

Android 10~13 Camera Pipeline 和软件结构介绍

文档版本
发布日期

V1.2
2023-01-05

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

Bluetooth®文字商标和徽标为蓝牙技术联盟的注册商标，紫光展锐（上海）科技有限公司对此类商标的任何使用均已获得许可。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

紫光展锐（上海）科技有限公司



前言

概述

本文档介绍了 Camera 系统的 Pipeline 结构，说明了 Pipeline 对 Camera 设计的重要性，并从软件分层的角度详细地描述了 Camera 软件系统整体结构。

读者对象

本文档适用于 Camera 开发人员。

缩略语

缩略语	英文全名	中文解释
AIDL	Android Interface Definition Language	Android 接口定义语言
CSI	Camera Serial Interface	相机串行接口
CPP	Camera Post Process	相机后处理
DCAM	Digital Camera	数码相机
HAL	Hardware Abstraction Layer	硬件抽象层
HIDL	Hardware Interface Definition Language	硬件接口定义语言
ISP	Image Signal Processor	图像信号处理
LWP	Light Weight Process	轻量级进程
OEM	Original Equipment Manufacture	原始设备制造商
VCM	Voice Coil Motor	音圈马达

符号约定

在本文中可能出现下列符号，每种符号的说明如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。

符号	说明
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.2	2023-01-05	增加 SC9832E 和 SC7731E 相关内容
V1.1	2022-07-28	增加 Android 13 相关内容。
V1.0	2022-07-07	第一次正式发布

关键字

Camera Pipeline、ISP、OEM、HAL Interface

目 录

1 Camera 系统架构	1
1.1 Camera 整体框架	1
1.2 Camera 软件系统	1
1.3 Camera 硬件 Pipeline	2
1.4 Camera 数据传输	6
2 Camera 软件架构	8
2.1 内核驱动层	10
2.2 硬件抽象层	10
2.2.1 HAL Interface	10
2.2.2 LWP	13
2.2.3 OEM	14
2.2.4 ISP 中间件	15
2.3 应用框架层	17
2.4 Open Camera 流程举例	19
3 参考文档	23

图目录

图 1-1 Camera 整体框架示意图	1
图 1-2 Android Camera 系统框架图 ^[1]	2
图 1-3 Android 硬件 Pipeline 结构 ^[1]	3
图 1-4 Camera 硬件图像处理 Pipeline	4
图 1-5 SC9832E Camera 硬件图像处理 Pipeline	5
图 1-6 SC7731E Camera 硬件图像处理 Pipeline	5
图 1-7 Snapshot 模式下数据流图	6
图 1-8 Preview 模式下数据流图	6
图 1-9 Video Recording 模式下数据流图	7
图 2-1 Android 体系结构图	8
图 2-2 Camera 软件体系结构图	9
图 2-3 DCAM/ISP 驱动结构	10
图 2-4 camera3_device_ops_t 调用时序图 ^[1]	12
图 2-5 数据通路配置结构图	13
图 2-6 LWP 后处理算法 Pipeline 链路结构图	14
图 2-7 OEM 功能结构图	15
图 2-8 ISP 的 Param Manager 和 3A 控制模块图	16
图 2-9 Android 13 ISP 的 Param Manager 和 3A 控制模块图	16
图 2-10 Framework 中 C/S 模式结构图	17
图 2-11 Framework 接口远程调用关系图 ^[1]	18
图 2-12 CameraX Frameworks 远程调用关系图 ^[1]	19
图 2-13 App 到 Camera Service 的打开摄像头流程图	20
图 2-14 Camera Service 到 HAL Interface 的打开摄像头流程图	21
图 2-15 HAL Interface 到 HAL 的打开摄像头流程图	22

表目录

表 2-1 camera3_device_ops_t 接口	11
表 2-2 AIDL 接口说明	18
表 2-3 HIDL 接口说明	19

1 Camera 系统架构

1.1 Camera 整体框架

Camera 系统的 Pipeline 如图 1-1 所示，其中涉及如下软件和硬件。

- 软件

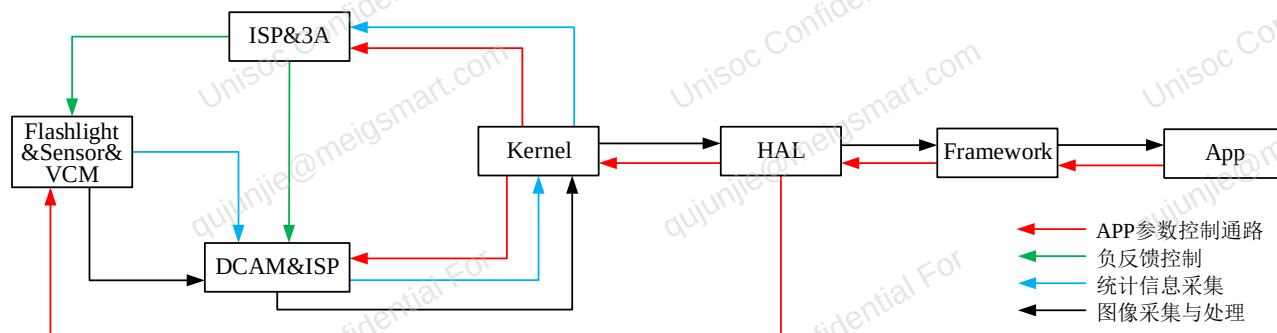
软件包括四个部分：

- App：作为体验终端，控制实际应用场景的切换及图片信息的显示。
- HAL（Hardware Abstraction Layer，硬件抽象层）和 Framework：起到通道的作用，要与硬件环境进行交互，同时实现与其他应用程序之间的交互。
- Kernel：用于管理与控制硬件设备流程及数据通信。
- ISP（Image Signal Processor，图像信号处理）：结合 App 下发的场景参数和硬件采集的统计信息，调整硬件设备参数，使硬件设备能够达到稳定且最优的采集数据状态。

- 硬件

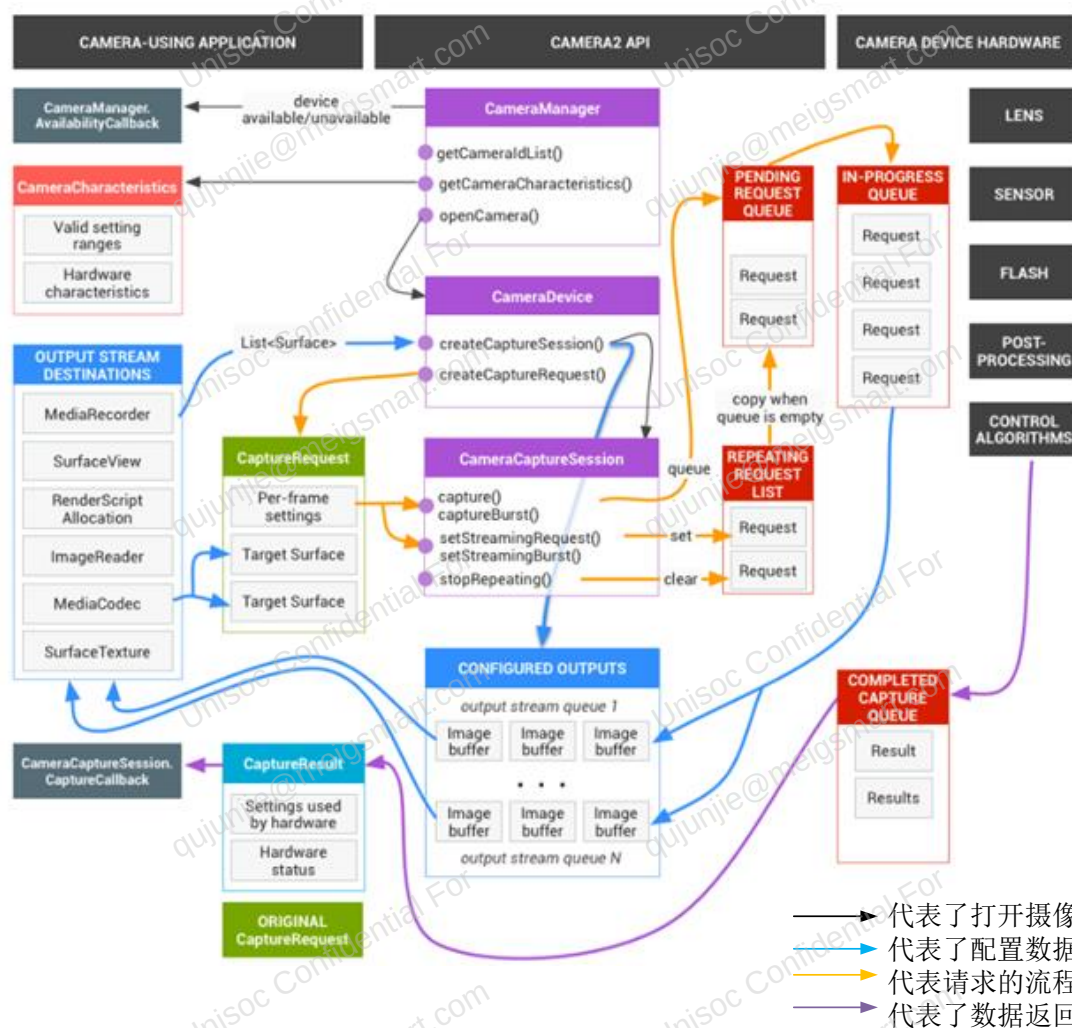
硬件包括闪光灯（Flashlight）、Sensor、VCM（Voice Coil Motor，音圈马达）、DCAM（Digital Camera，数码相机）和 ISP 等模块。

图1-1 Camera 整体框架示意图



1.2 Camera 软件系统

Android 平台的 Camera 系统框架如图 1-2 所示。

图1-2 Android Camera 系统框架图^[1]

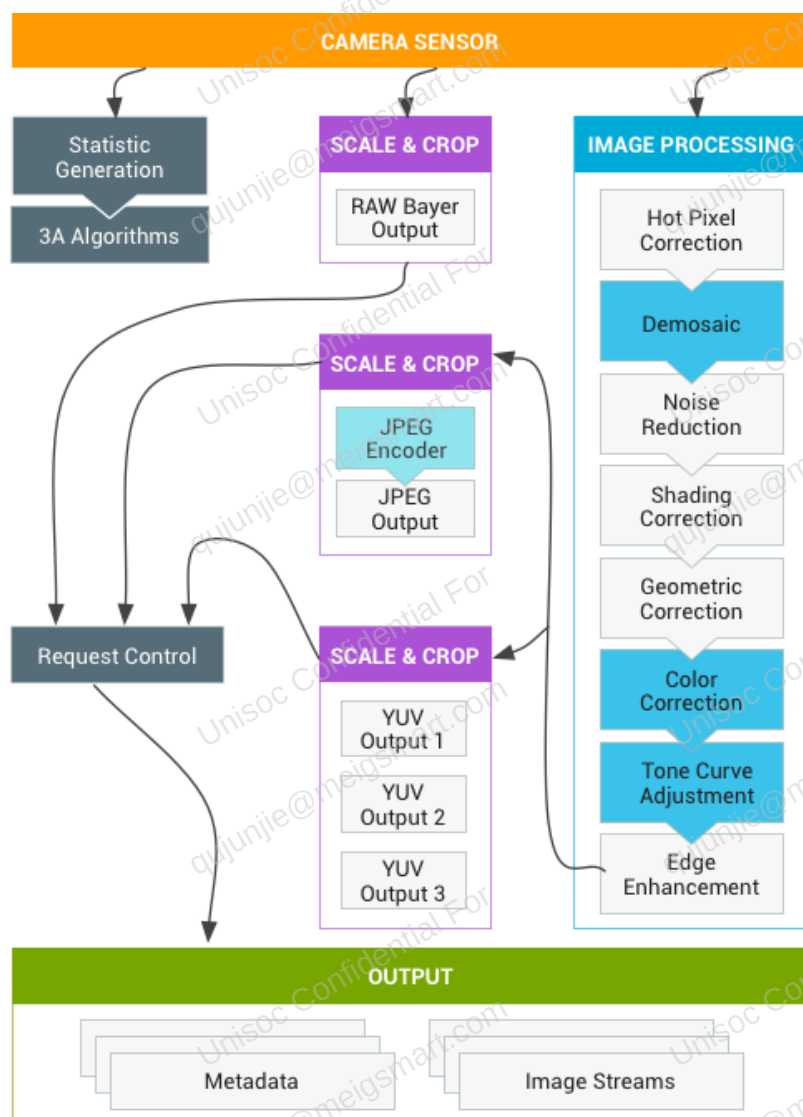
应用框架层会针对捕获的结果向 Camera 子系统发出请求，一个请求对应一个结果。请求包含有关捕获结果和处理这些结果的所有配置信息，其中包括分辨率、像素格式、手动传感器、镜头、闪光灯控件、3A 操作模式、RAW 到 YUV 处理控件以及统计信息的生成。一次可以发起多个请求，请求时不会出现阻塞，请求始终按照接收的顺序处理。

1.3 Camera 硬件 Pipeline

Android 给出了硬件 Pipeline 结构信息如图 1-3 所示，Sensor 根据输出数据的特点将其分为统计数据 and 图像数据，这两类数据都可以返回 HAL 层使用。

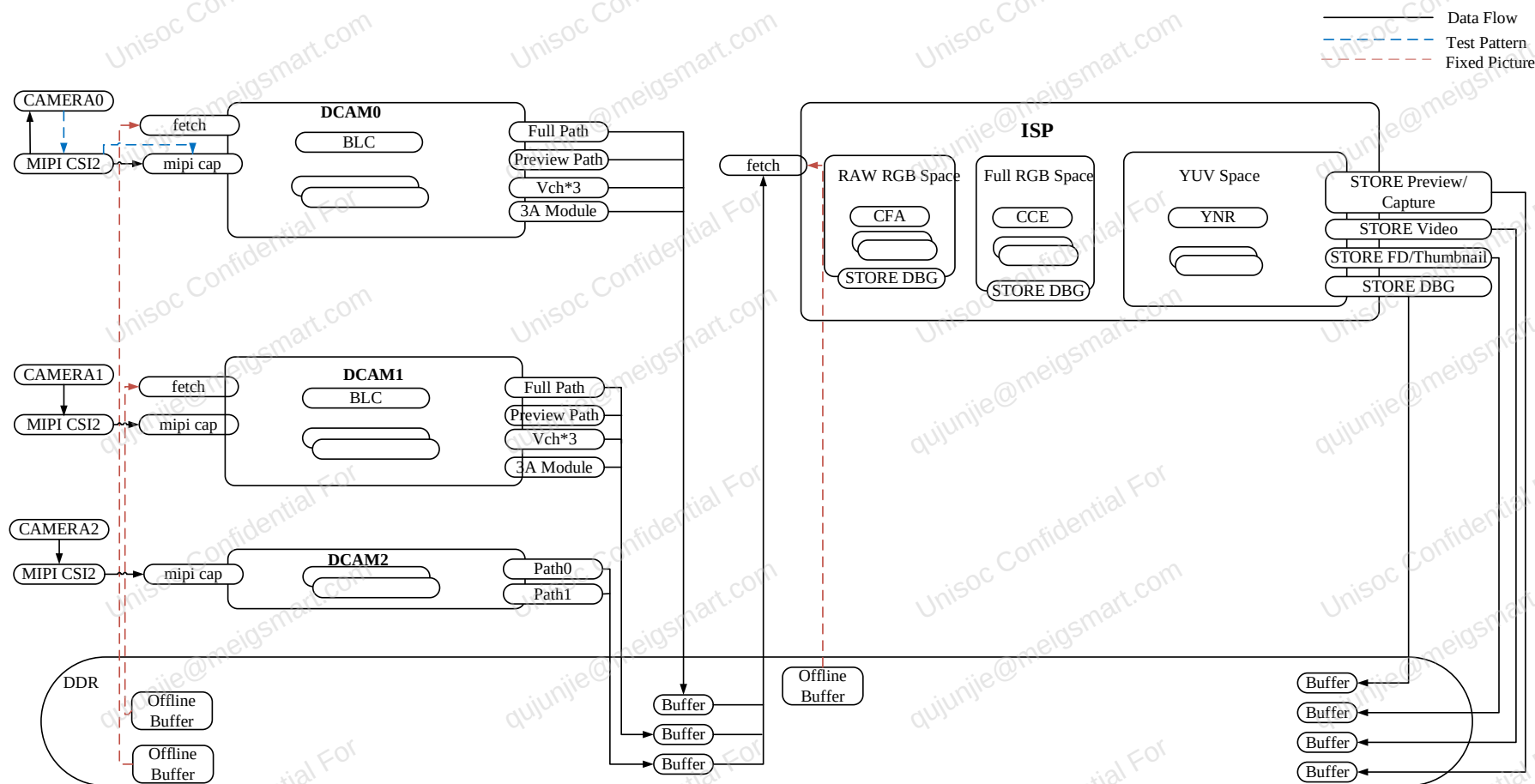
- 统计数据主要用于 3A 算法的计算，进而形成图 1-1 中闭环控制。
- 图像数据主要分为两类，RAW 数据和经过 ISP 处理后的 YUV 数据。

图1-3 Android 硬件 Pipeline 结构^[1]



展锐设计的硬件图像处理 Pipeline 如所图 1-4 示。

图1-4 Camera 硬件图像处理 Pipeline



图像处理的核心在 DCAM 和 ISP。从 Sensor 获取的 RAW 图数据，经过 DCAM Pipeline 的处理后缓存至 DDR 存储器中，ISP 获取需要处理的图像数据经过 RAW RGB 域、full RGB 域，最后到 YUV 域生成图像数据。将该处理后的图像数据提供给 HAL 层，进行后处理或者编码操作。

- DCAM 的特点如下：
 - Sensor 获取的数据直接提供给 DCAM，是一种在线的方式。

DCAM 可以离线对 RAW 数据进行处理。

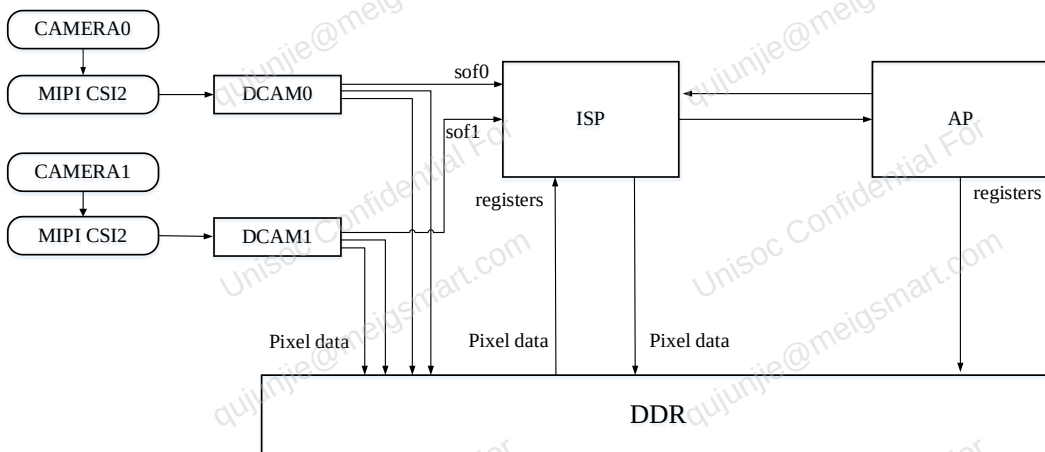
- ISP 的特点如下：
 - ISP 的工作方式是离线的，不存在数据溢出的问题。
 - ISP 可以输出多种图像数据，ISP 的输出通路包括：Preview/Capture、Video、FD（Face Detection，人脸识别）/Thumbnail 及 Debug 接口。可以满足上层对图片大小的不同需求。

说明

- 对于 SC983E 芯片，Sensor 数据经过 DCAM online 处理输出到 DDR，紧接着软件通过调用 ISP 将数据从 DDR 中读出来，才算完整处理完一帧数据。ISP 的工作方式是离线的，不存在数据溢出的问题。
- 对于 SC7731E 芯片，Sensor 数据经过 DCAM 处理直接输出到 ISP，全程以 online 的形式完整处理完一帧数据。

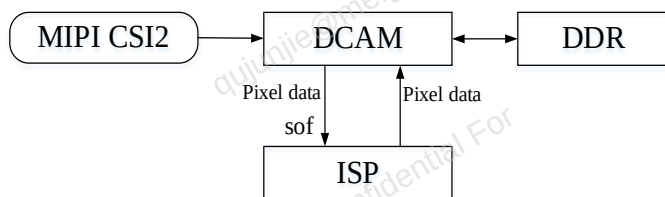
SC9832 芯片平台 Camera 硬件处理流程的 Pipeline 的示意图如图 1-5 所示。

图1-5 SC9832E Camera 硬件图像处理 Pipeline



SC7731E 芯片 Camera 硬件处理流程的 Pipeline 的示意图如图 1-6 所示。

图1-6 SC7731E Camera 硬件图像处理 Pipeline



1.4 Camera 数据传输

Camera 基础的三个功能是拍照（Snapshot）、预览（Preview）及录像（Video Recording），数据流图分别如图 1-7、图 1-8 及图 1-9 所示。

图1-7 Snapshot 模式下数据流图

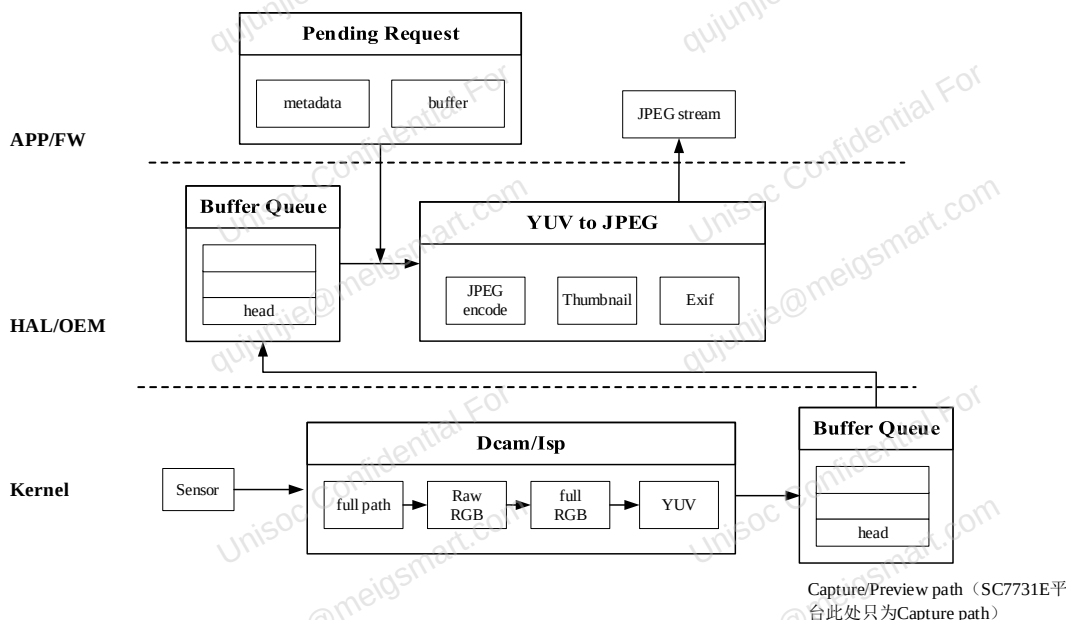


图1-8 Preview 模式下数据流图

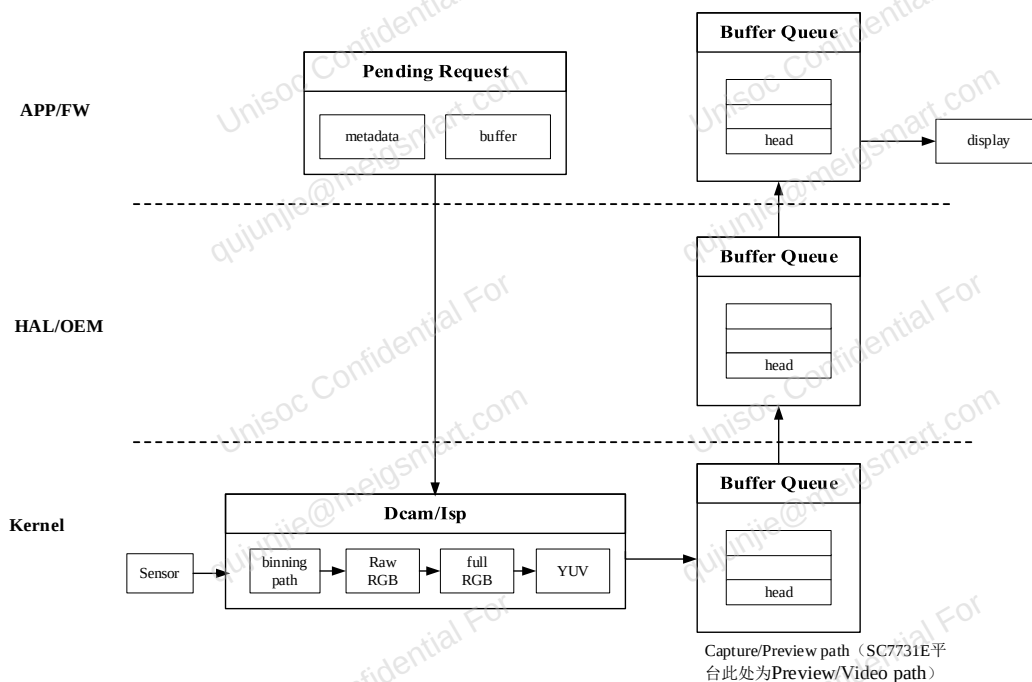
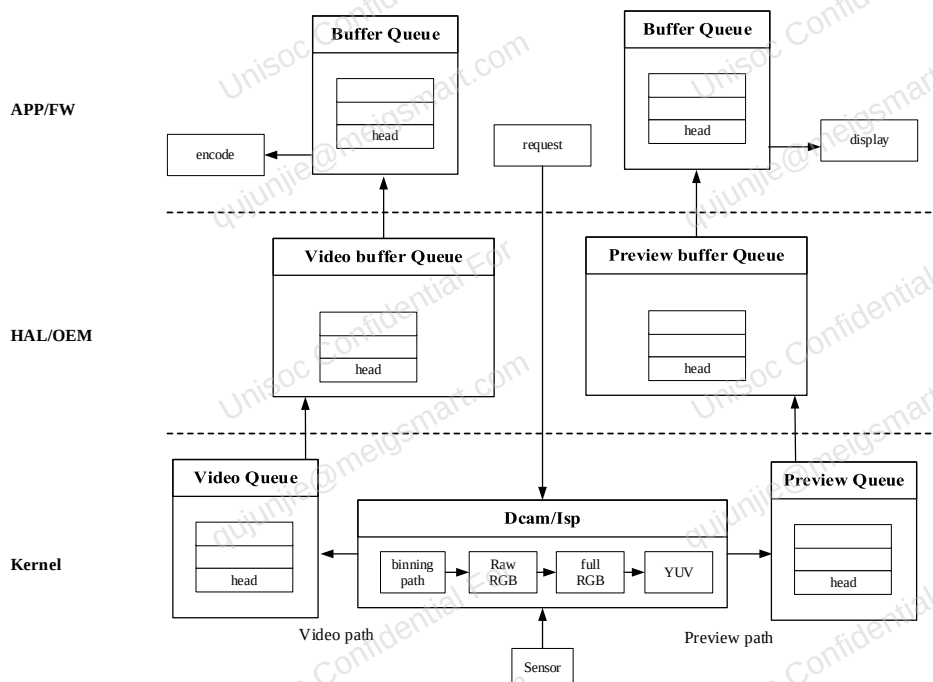


图1-9 Video Recording 模式下数据流图



- Snapshot 与 Preview 差异如下:

- 在硬件方面, DCAM 上 Snapshot 需要 full path 处理, preview 只需要 binning path 处理。
- 在软件方面, Snapshot 通过硬件上传的图片数据需要进行 JPEG 编码, 但是 Preview 只需要 YUV 格式图片信息用于上层显示即可。

- Video Recording 与 Preview 差异如下:

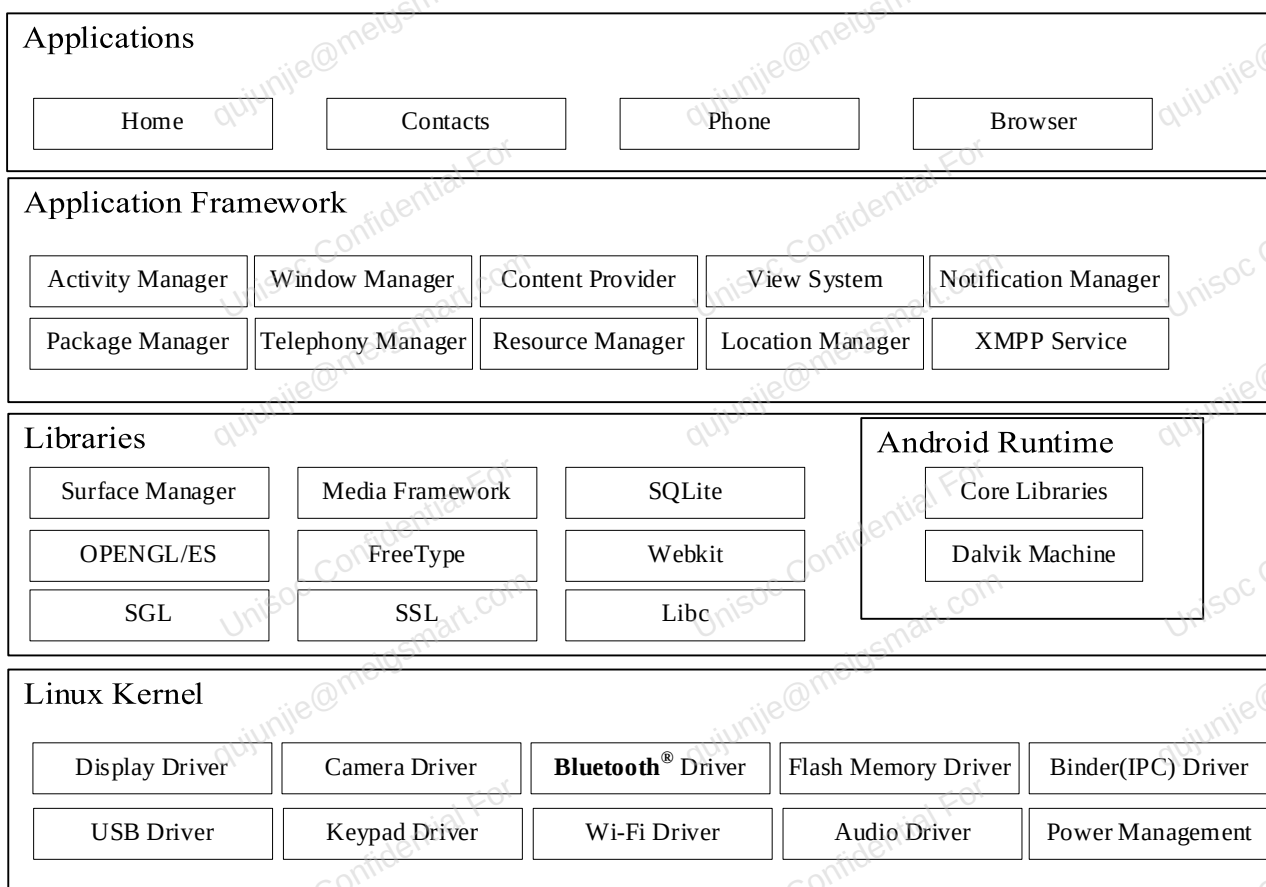
Video Recording 和 Preview 通路上进行的后处理内容存在差异, Video Recording 用于编码成视频, Preview 用于显示, 二者可以同时存在, 在 buffer 分配等方面存在着不同。

通过比较差异, 发现不可能通过一种 Pipeline 的设计来满足多种功能需求, 硬件和软件在设计上满足多样性的同时, 也要保证 Pipeline 处理图片的效率, 需要对整体 Pipeline 有着深入的理解, 从结构上保证多样性和运行效率。

2 Camera 软件架构

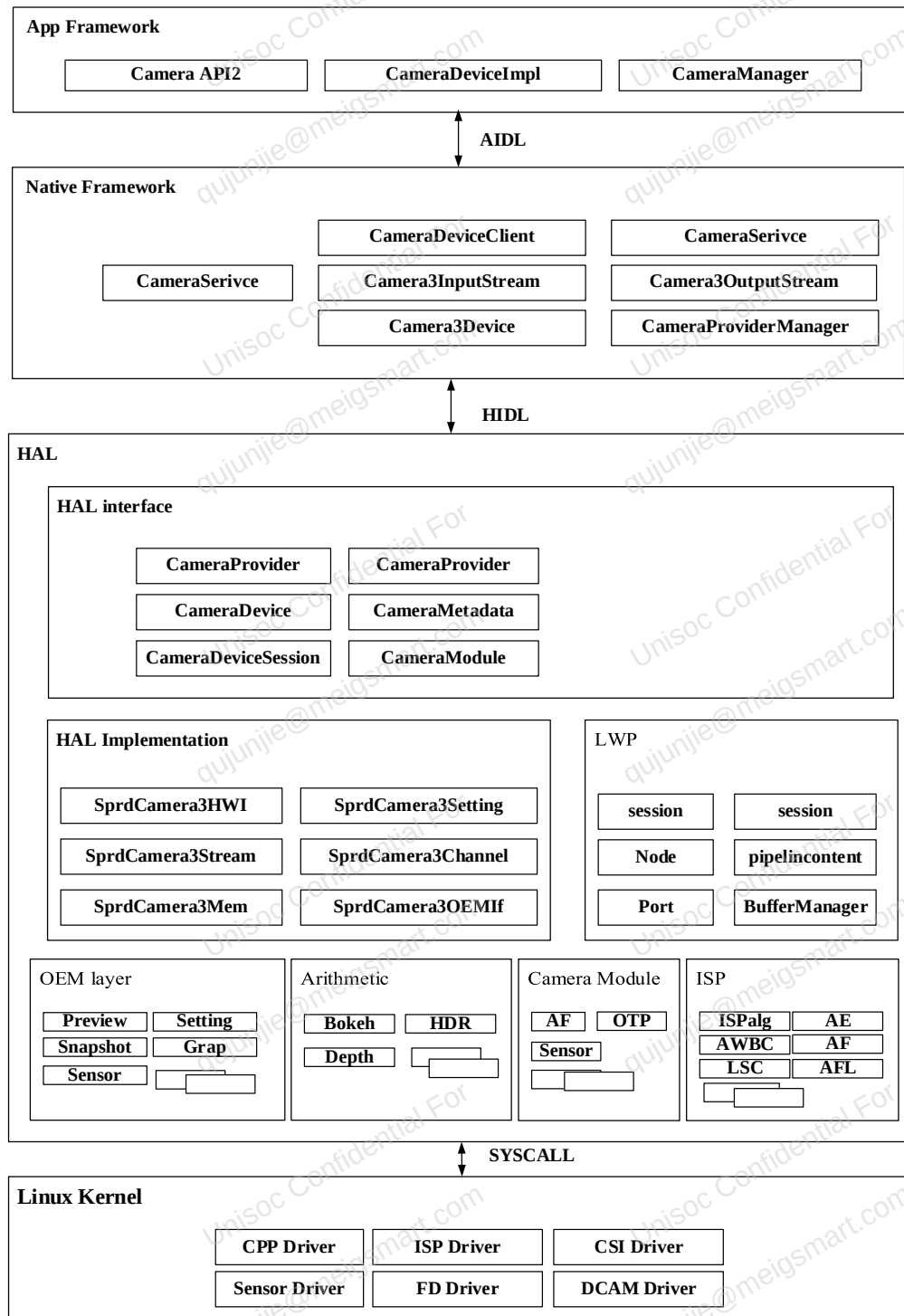
Camera 系统软件是基于 Android 体系结构构建的。如图 2-1 所示，Android 作为一个移动设备的平台，软件系统主要由四层结构即应用程序（Applications）、应用框架层（Application Framework）、各类库（Libraries）和运行环境（Android Runtime）及操作系统内核（Linux Kernel）组成。下文基于 Android 体系结构介绍展锐平台的 Camera 软件架构的具体实现。

图2-1 Android 体系结构图



以 Android 体系结构为基础，图 2-2 详细展示了展锐平台的 Camera 软件体系结构，分为 App Framework 层、Native Framework 层、HAL 层以及 Linux Kernel 层，图中详细地展示了每层可划分的功能模块和层与层之间的交互方式。

图2-2 Camera 软件体系结构图



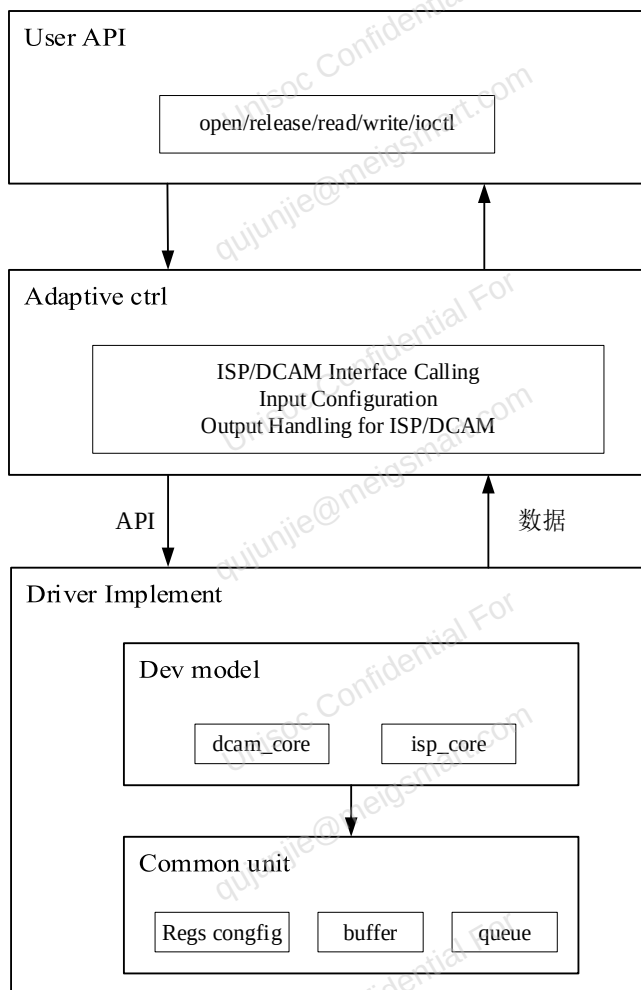
说明

- Android 13 及后续版本新增了 LWP 架构。
- SC9832E 和 SC7731E 芯片平台没有 LWP 架构。

2.1 内核驱动层

图 2-2 中 Linux Kernel（即内核驱动层）将硬件模块分为 DCAM、ISP、Sensor、CPP（Camera Post Process，相机后处理）、CSI（Camera Serial Interface，相机串行接口）及 FD 模块。上层调用 Kernel 驱动主要采用系统调用方式。DCAM/ISP 驱动结构调用方式如图 2-3 所示，native 层软件通过系统调用 open/release/ioctl 等用户接口，实现对 DCAM/ISP 硬件设备的控制与数据交互，Driver Implement 调用各硬件设备的驱动接口来调用 DCAM/ISP 各自驱动流程单元，从而实现不同的 Pipeline 处理流程。

图2-3 DCAM/ISP 驱动结构



2.2 硬件抽象层

硬件抽象层包含 HAL Interface、LWP（Light Weight Process，轻量级进程）、OEM 和 ISP 四个子模块。

2.2.1 HAL Interface

HAL Interface 中定义了 Camera Service 的标准接口。在 HAL 层中 Camera Provider 主要用来处理 Camera HAL 的功能，Camera Provider 需要与 Camera Service 进行跨进程间的通信，Android 上采用 HIDL

(Hardware Interface Definition Language, 硬件接口定义语言), 即一种 binder IPC 的方式实现进程间通信。

不同的硬件厂商在实现 HAL 方面有一定的差异, 主要有两个功能, 实现 HAL Interface 和构建数据通路。HAL Interface 实现 Android 标准接口 camera3_device_ops_t 中所定义的接口, 每个接口的功能如表 2-1 所示。

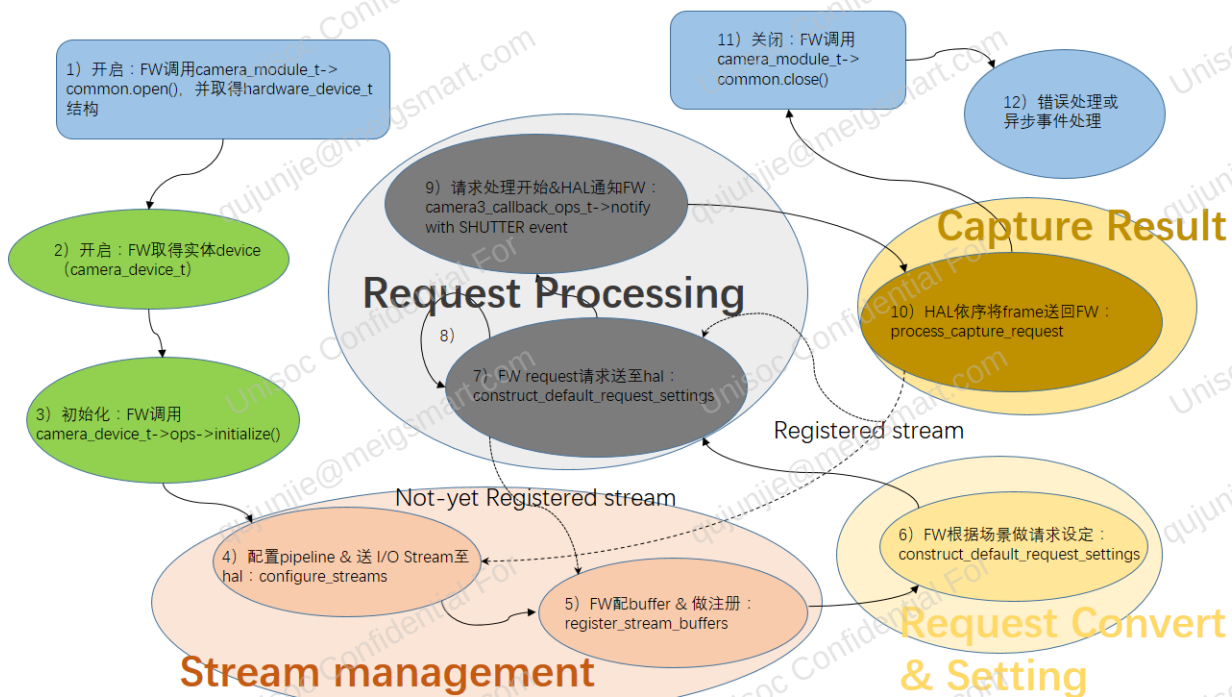
表2-1 camera3_device_ops_t 接口

接口	功能
initialize	状态初始化。
configure_streams	向 HAL 层设备传递了输入输出流信息。
register_stream_buffer	Framework 层分配 gralloc buffer。
construct_default_request_settings	标准的 Camera 用例创建拍照设置, 设备必须返回一个与所要求的用例相匹配的 buffer 设置。
process_capture_request	处理上层请求。
get_metadata_vendor_tag_ops	获取方法查询供应商扩展的元数据信息。
dump	输出 Camera 设备的调试状态。
flush	清除某个设备上当前正在处理的所有拍照请求和 Pipeline 中的所有 buffer。

如图 2-4 所示, camera3_device_ops_t 调用大致分为 6 个过程, 分别是:

1. Device Initialize: 对应图中 1)、2)、3)。
2. Stream Management
3. Request Convert & Setting
4. Request Processing
5. Capture Result
6. Device Close: 对应图中 11)、12)。

图2-4 camera3_device_ops_t 调用时序图^[1]

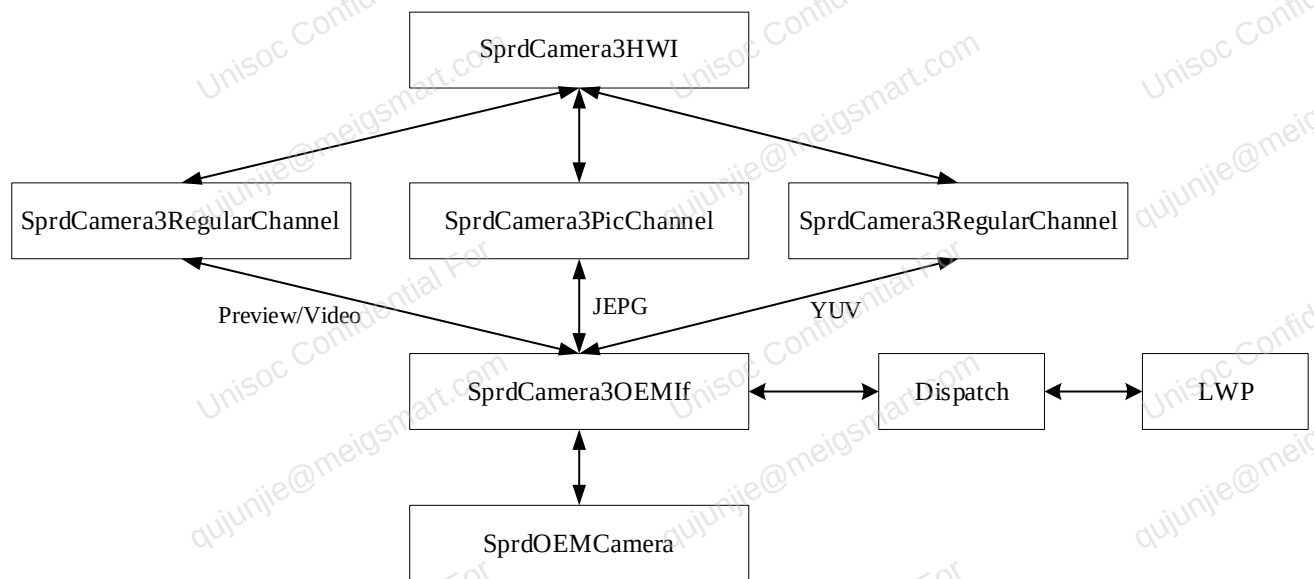


调用流程接口分别描述如下:

- **Stream Management** 作为 HAL 层的重要接口为预览、拍照、参数及回调数据建立相应的通道并添加相应的 Stream。Stream Management 主要包括两种 Stream:
 - `Camera3OutputStream` 用于 HAL 填充上报的数据信息。
 - `Camera3InputStream` 用于 Framework 填充 Input Stream 并回调下发给 HAL。
- **Request Convert & Setting** 将所有的 Stream 都打包成 Request，根据 Stream 的不同，从而构建不同类型的请求。
- **Request Processing** 处理由 Framework 发送给 HAL 的请求，根据不同的场景改变请求方式。
- **Capture Result** 将底层处理后的数据进行封装，封装成 Framework 识别的数据结构，并上报给 Framework 层解析与处理。

数据通路配置结构如图 2-5 所示。

图2-5 数据通路配置结构图



其中：

- SprdCamera3HWI 为上层接口实现类。
- SprdCamera3RegularChannel 为 Preview/Video 数据通路实现类，主要输出 YUV 数据格式图片信息。
- SprdCamera3PicChannel 为 Capture 数据通路实现类，主要输出 JPEG 数据格式图片信息。
- SprdCamera3OEMIf 为与 OEM 通信的接口类。
- Dispatch 模块处于 camera Hal 层，负责分发 request 和 match 帧信息，实现与 LWP 模块的交互。

说明

- SC9832E 和 SC7731E 芯片平台没有 LWP 架构，所以数据通路配置结构中不存在 Dispatch 和 LWP 模块。

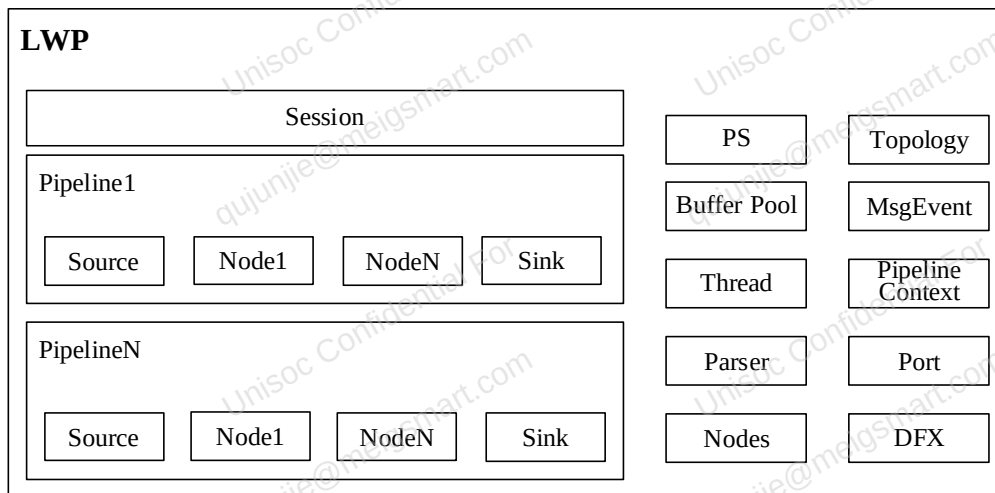
2.2.2 LWP

LWP 作为一个完整的模块和现有的展锐 Camera Android Hal 对接，对接模块为 Dispatch。主要功能为：

- 接收 Dispatch 设置当前模式 pipeline 链路构建要求
- 对应 pipeline node 实例化
- pipeline buffer 管理
- pipeline 线程管理
- 预览/录像/拍照后处理接受处理
- 回调
- 快停快切功能等

LWP 的功能结构如图 2-6 所示。

图2-6 LWP 后处理算法 Pipeline 链路结构图



其中：

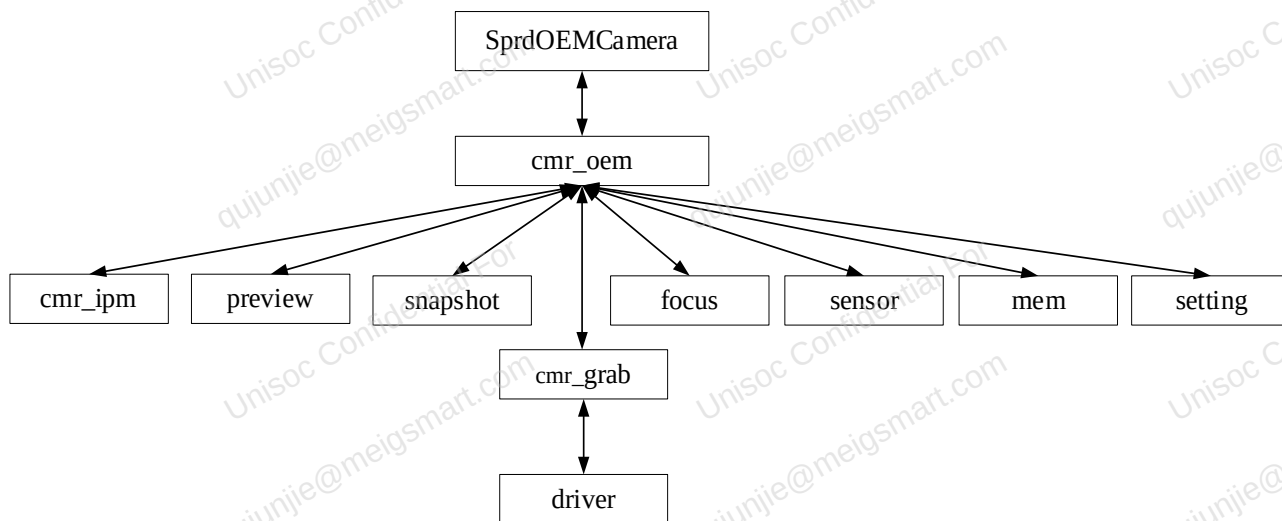
- **Session:** 作为 LWP 的主控模块，将根据外围条件找到配置文件中的某个配置完成 Pipeline 的选择，并告知各个 Pipeline 配置信息，接受外围控制信息完成对各子模块的控制，实现数据状态对外围的回调。
- **Topology:** 这个模块主要将配置文件解析为 LWP 的内部定义的结构体，Session 将会将这些配置信息给到相应的内部模块，即 Pipeline。
- **PS (Parameter Service):** 实现外围和内部参数的管理，对各硬件各数据流各数据帧的参数精确管理，以便实现数据流的帧帧控制。
- **Pipeline Context:** 管理多种类型的 Pipeline，通过 Session 配置的 Pipeline 的配置信息构建 Pipeline 的拓扑结构，根据 session 的控制管理 pipeline 的控制逻辑。
- **MsgEvent:** 实现各个内部模块之间的消息通讯。
- **Thread 和 Buffer Pool:** 为 LWP 提供统一的线程和 Buffer 管理，方便使用和 debug。
- **Nodes:** 实现具体的数据控制处理。
- **Port:** 实现数据的流转，为各个 Node 间搭建联系。
- **DFX:** 完成对 Pipeline 图像、性能和内存等关键性指标监控。

2.2.3 OEM

OEM 层向上提供接口被 HAL Implementation 调用，向下调用驱动接口完成对设备的控制。Camera OEM 层主要功能为：接收上层设置的相机参数、Camera 初始化、启动预览、停止预览、普通拍照、ZSL 拍照、停止拍照、对焦以及将数据和状态通知到 HAL 层等。

OEM 的功能结构如图 2-7 所示，其中 cmr_grab 主要与驱动交互并获取驱动提供的数据信息，cmr_oem 主要提供 SprdOEMCamera 调用的功能接口并与其他功能模块进行交互。

图2-7 OEM 功能结构图



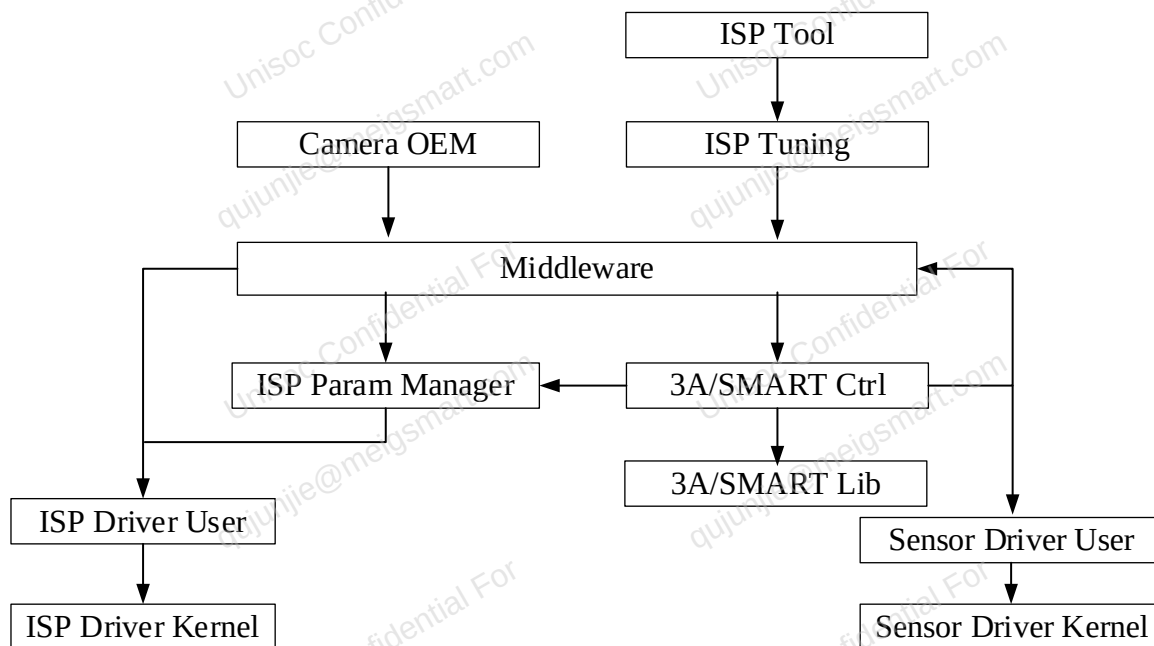
2.2.4 ISP 中间件

ISP 中间件由 Param Manager 和算法库模块组成。算法库模块分为图像后处理算法和 3A 算法。

Param Manager 控制模块用于 ISP tuning 参数管理和 3A 状态控制，作为 ISP 的辅助功能来实现对硬件子模块参数的集中管理与控制，如图 2-8 所示：

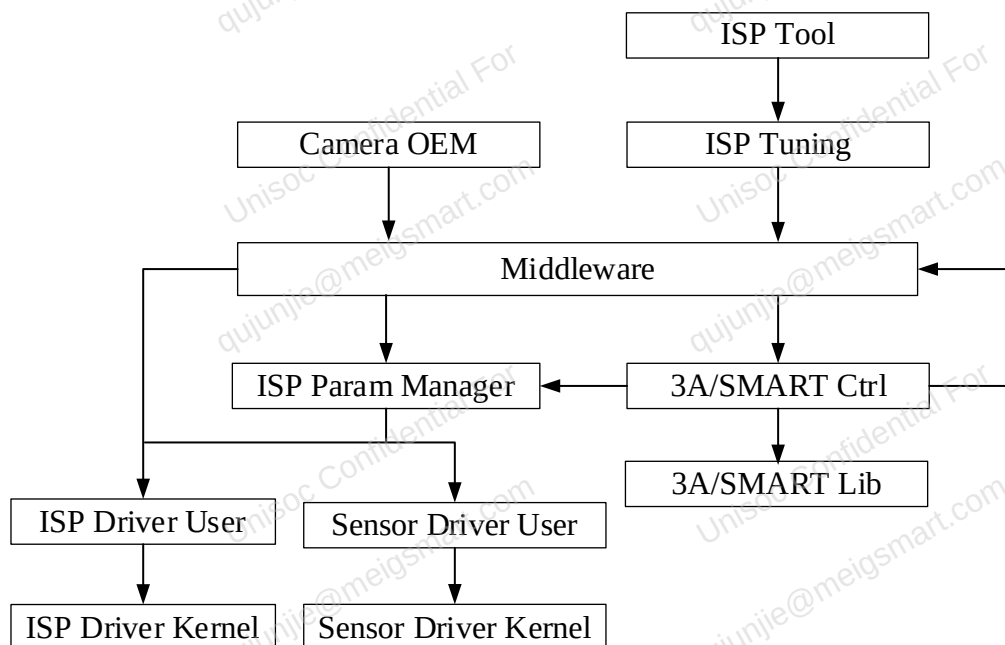
- Middleware 层主要用于 Camera OEM 层对 ISP Param Manager 和 3A/SMART Ctrl 控制交互。
- OEM 层通过 Middleware 从 Param Manager 模块获取参数。
- Middleware 从 ISP tuning 模块获取参数并通过 Param Manager 模块更新相关硬件模块信息。
- 3A 模块通过 Param Manager 模块实时更新硬件模块信息。

图2-8 ISP 的 Param Manager 和 3A 控制模块图



Android 13 平台的 ISP 的 Param Manager 和 3A 控制模块略有不同，如图 2-9 所示。

图2-9 Android 13 ISP 的 Param Manager 和 3A 控制模块图



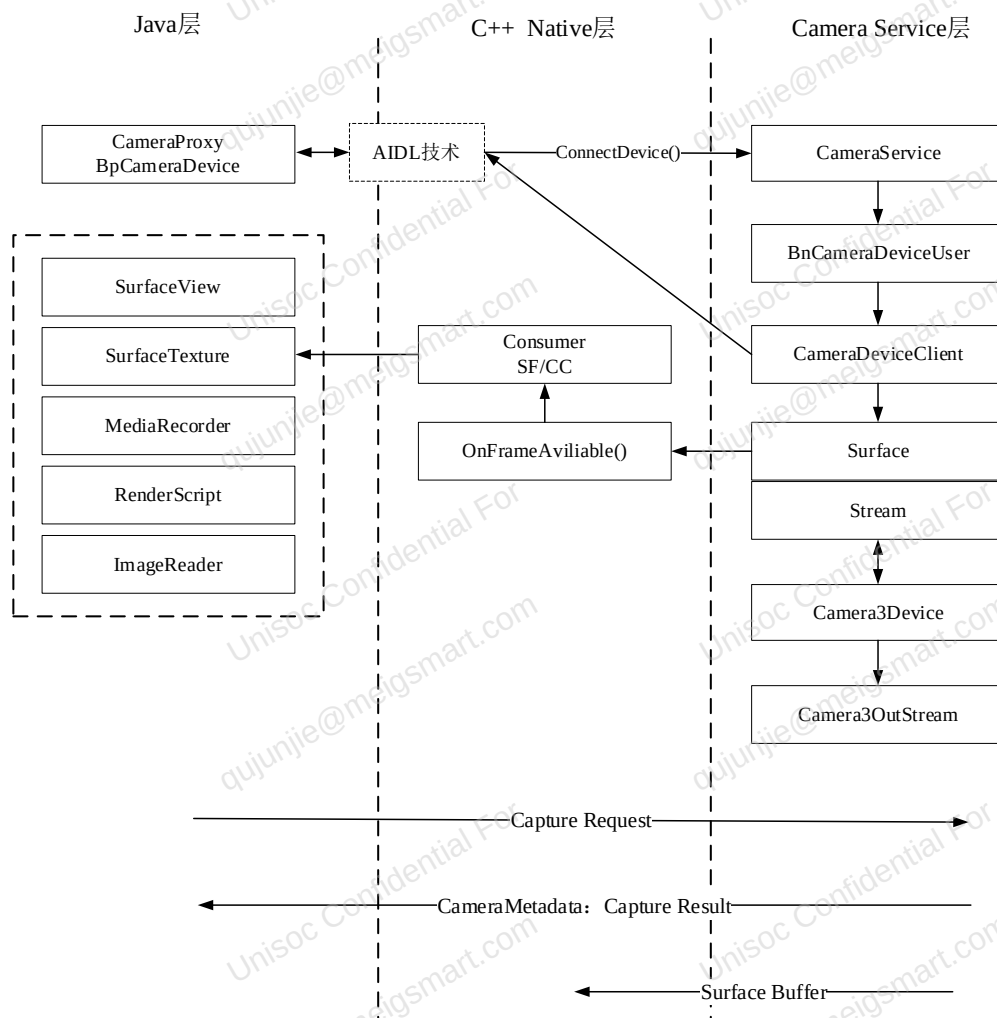
2.3 应用框架层

Camera 的 Framework 层分为：

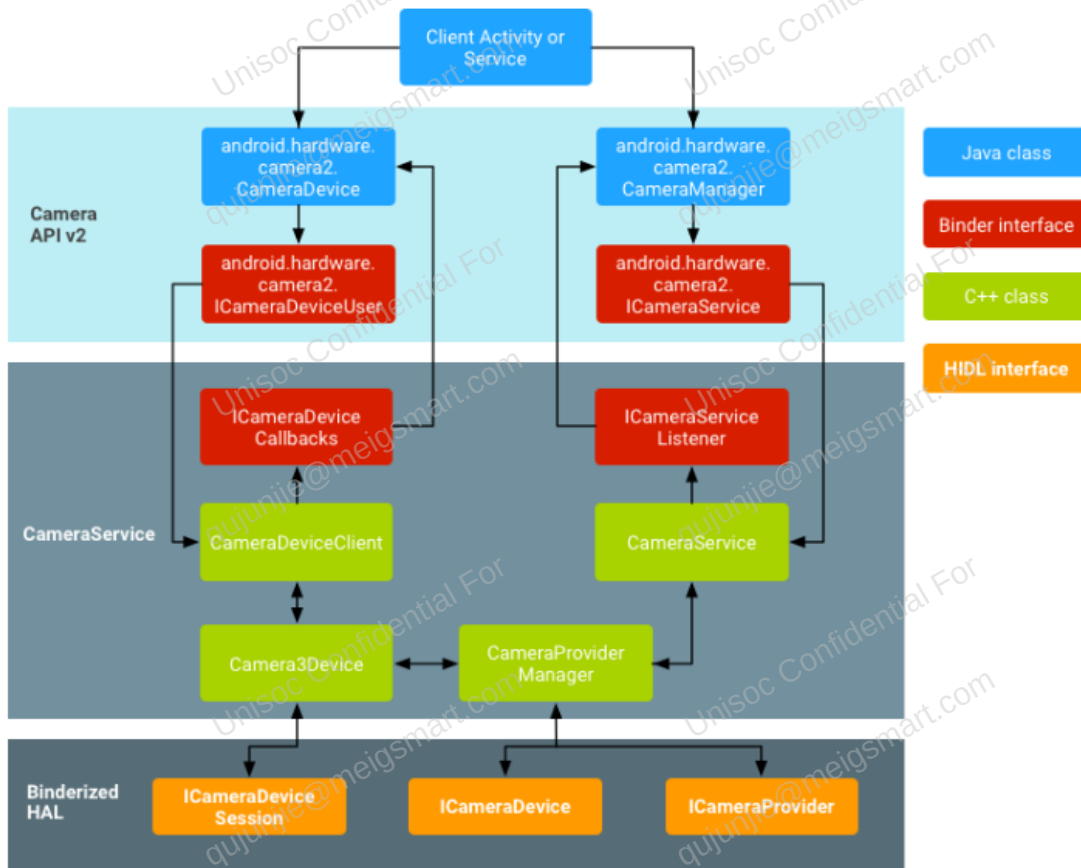
- App Framework 主要提供 Camera API 接口供应用调用。
- Native Framework 主要与 HAL 进行通信，完成场景切换和获取底层上报的数据信息。

如图 2-10 所示，每一个使用 Camera 的终端，如 SurfaceView、SurfaceTexture、ImageReader 等都是生产者，CameraService 端是消费者，利用这样的生产者/消费者模型，建立 Camera 客户端和服务端端的连接以及 buffer 数据的传递，图中展示了 Capture Request、CameraMetadata 和 Surface Buffer 的流转关系。

图2-10 Framework 中 C/S 模式结构图



如图 2-11 所示，在 Framework 层中，层与层之间的通信是跨进程的。

图2-11 Framework 接口远程调用关系图^[1]

主要接口有 AIDL（Android Interface Definition Language，Android 接口定义语言）接口和 HIDL 接口，其中 AIDL 接口即为图 2-11 中红色的 Binder interface。AIDL 可以允许 App Framework 代码访问 Native Framework 的接口，定义了 ICameraService、ICameraDeviceUser、ICameraServiceListener 及 ICameraDeviceCallback 接口，具体功能请参考表 2-2。

表2-2 AIDL 接口说明

接口	作用
ICameraService	相机服务接口，用于请求连接、添加监听等。
ICameraDeviceUser	已打开的特定相机设备的接口，App Framework 可通过它访问具体设备。
ICameraServiceListener	是 Camera Service 到应用框架的监听。
ICameraDeviceCallback	是 Camera Device 到应用框架的回调。

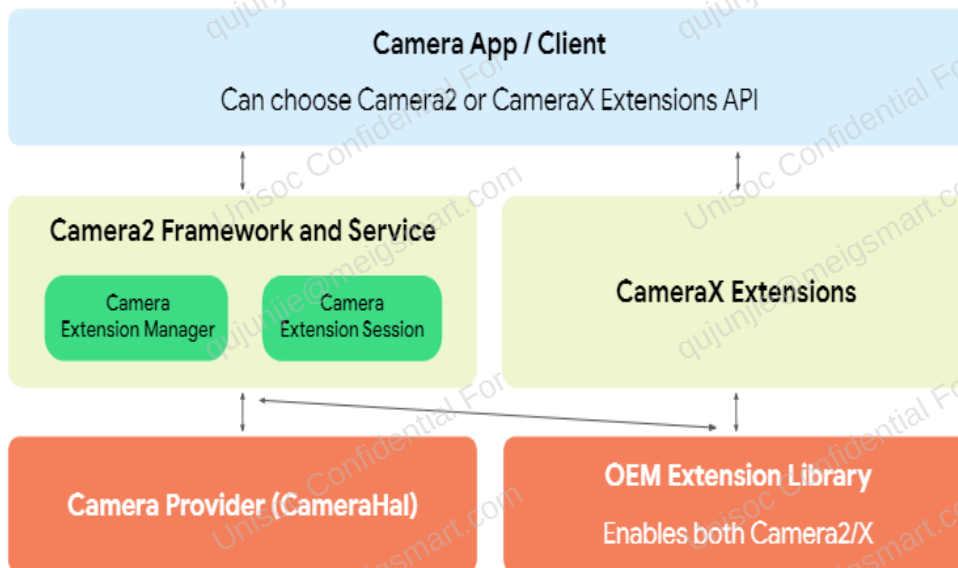
系统启动的时候，会从 Camera Server 进程中分离出来一个独立进程的 Camera Provider 服务，其名称为 android.hardware.camera.provider@2.4-service，用来控制 Camera HAL。Camera Server 通过 HIDL 机制与 Camera Provider 进行通信。HIDL 主要功能是将 Service 与 HAL 进行隔离，进而方便 HAL 部分进行独立升级。Framework 定义了三个接口，分别为 ICameraDeviceSession、ICameraDevice 和 ICameraProvider，具体功能请参考表 2-3。

表2-3 HIDL 接口说明

接口	作用
ICameraDeviceSession	活跃的 Camera 设备会话接口
ICameraDevice	Camera 设备接口
ICameraProvider	用于枚举单个设备并管理其状态

Android 13 上为了方便第三方相机应用扩展使用，CameraX 提供了一个 Extensions API，用于使用制造商在各种 Android 设备（手机、平板电脑等）上实现特效。一个供应商的相机扩展实现同时支持 Camera2 和 CameraX 框架，如图 2-12 所示。

图2-12 CameraX Frameworks 远程调用关系图^[1]



- 通过 CameraX 框架来使用扩展功能，比如：美颜、HDR 等。CameraX 提供一个 Camera Extensions Manager 类，可以确定设备是否支持某一特定的扩展模式，并为其获取一个启用扩展的 Camera Selector。使用 CameraX 可以更加简洁方便地使用扩展功能。缺点是不能灵活地控制 Camera，且每次拍照时只能使用一个扩展功能。
- 通过 Camera2 Framework and Service 来使用 bokeh、美颜、HDR 等功能。使用该方式可以灵活控制 Camera，除了使用平台自身的算法，还可以通过 Camera2 随意控制 request 和使用 result，并可以集成开发的算法。

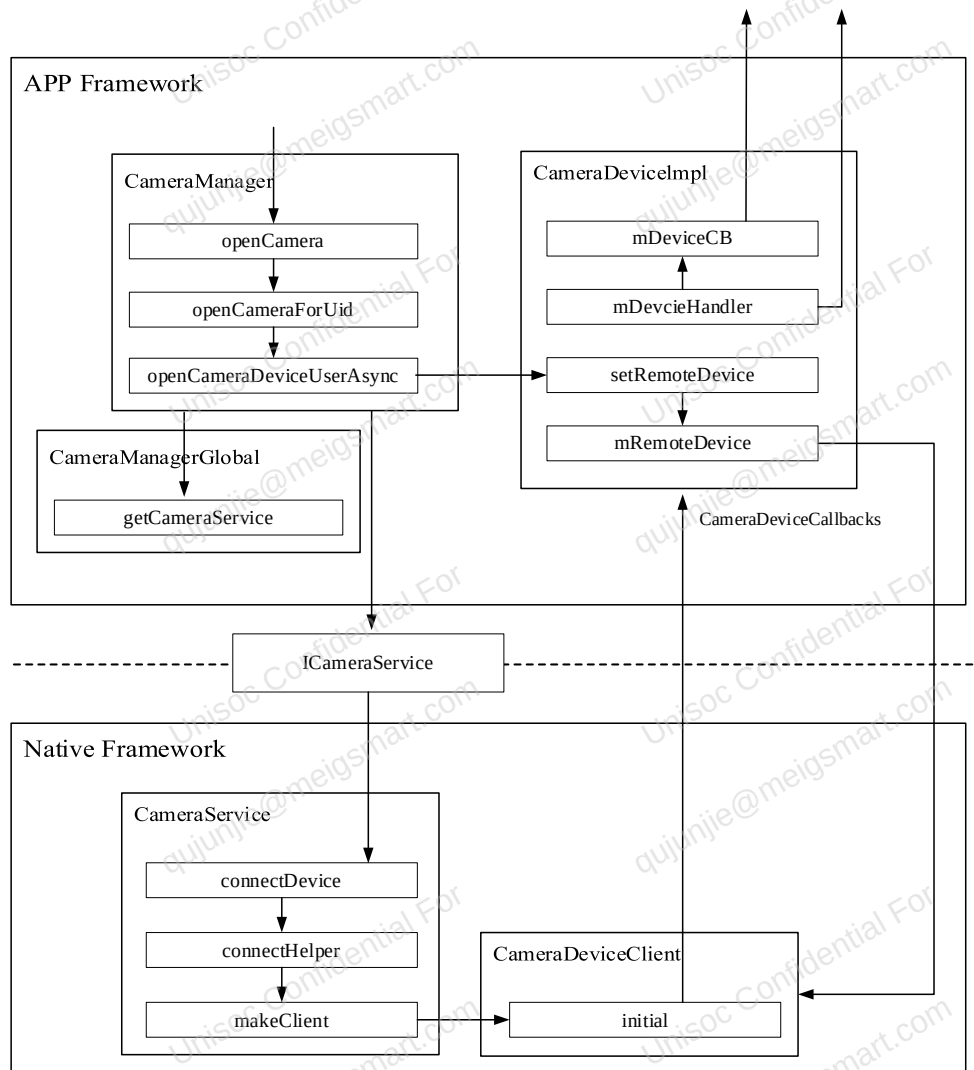
2.4 Open Camera 流程举例

Android 的 Camera 架构涉及多个层级间的通信与调用关系，以打开摄像头分析流程为例，按照 Camera 整体的软件架构自上而下进行解析。

1. Camera App 到 Camera Service

Camera App 到 Camera Service 涉及 App、App Framework 及 Native Framework。App 直接调用 App Framework 提供的接口，App Framework 需要通过 binder 远程调用 Native Framework 中 Camera Service 的接口，binder 通信实现方式为 AIDL。如图 2-13 所示，CameraManager 通过 ICameraService 接口调用 CameraService 中的 connectDevice 接口，其中 CameraDeviceImpl 向 APP 层输出信息。CameraDeviceClient 需要向上提供 CameraDeviceCallbacks 回调接口，以实现上下层之间的握手通信。

图2-13 App 到 Camera Service 的打开摄像头流程图

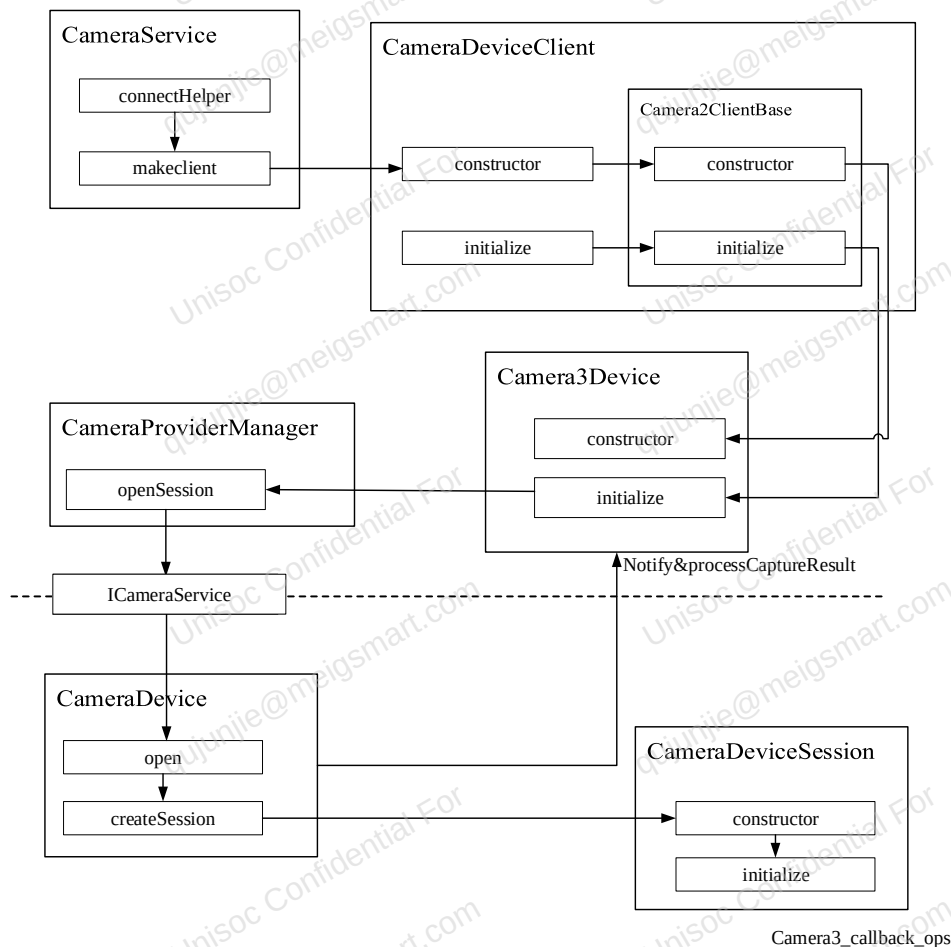


2. Camera Service 到 HAL Interface

Camera 通过 C/S 的架构实现客户端与 Camera 终端通信，在 Camera Server 会搜寻 Provider Service，并将其加入到内部的 CameraProviderManager 中进行统一管理。如图 2-14 所示，CameraService 对 Provider 中 HAL Interface 调用是一种跨进程的远程调用，其实现方式为 HIDL。Camera3Device 是 Framework 与 HAL 进行对接的实例，控制着所有的 Stream，其中主要有两种，分别是 Camera3OutputStream 和 Camera3InputStream。在通过 openSession 远程调用 CameraDevice 的 open 函

数过程中，CameraDevice 通过回调函数 Notify 和 processCaptureResult 向上层提供消息和上报数据。正常工作通路下，CameraDevice 管理输入输出数据流来实现数据的上下传递。

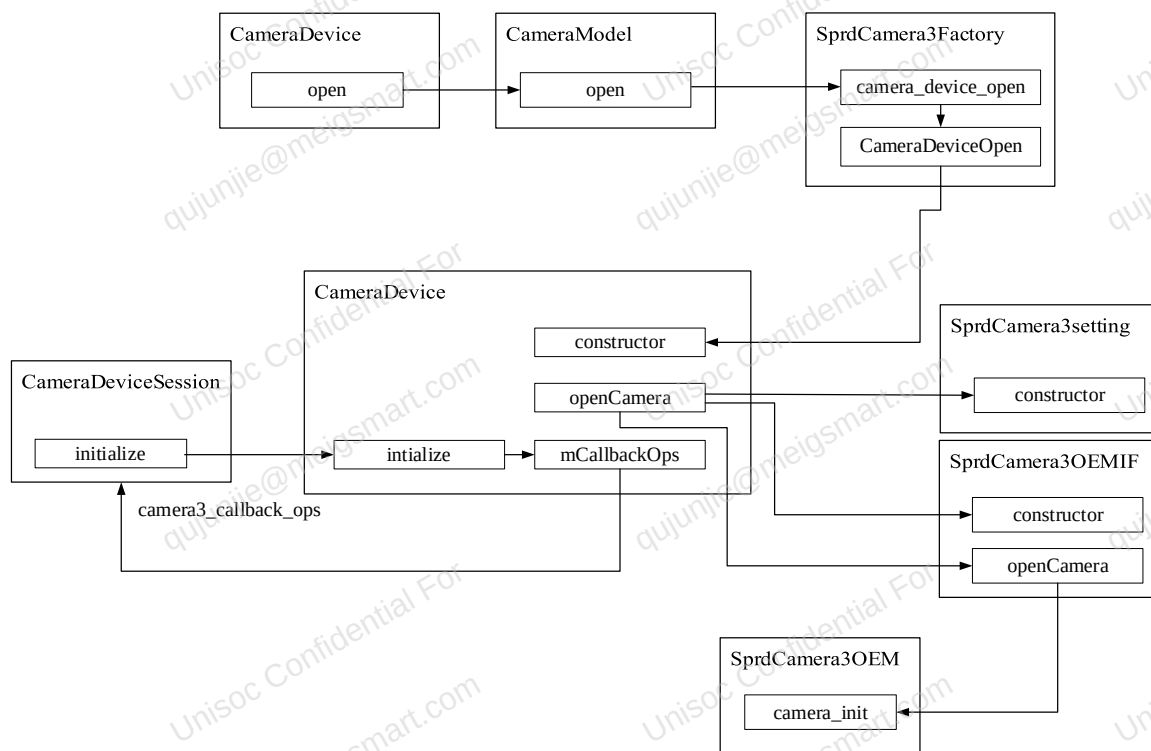
图2-14 Camera Service 到 HAL Interface 的打开摄像头流程图



3. Hal 层 open camera 实现方案

Android 在 HAL Interface 中定义了明确的接口功能，Camera HAL Implementation 承担着解析与实现 HAL Interface 定义的接口。如图 2-15 所示，CameraDevice 通过 CameraModule 向下构建 Camera 硬件设备打开通路，其中 SprdCamera3HWI 作为 HAL 接口实现类，连接底层与上层之间的通信，通过 openCamera 接口直接调用 SprdCameraOEMIf 中的 openCamera 函数，对 SprdCamera3OEM 中 camera_init 进行跨库接口调用。camera_init 是 OEM 层的接口，通过该接口可以一步步往下调用，最终调用到 Kernel 驱动完成对硬件设备的初始化，完成打开摄像头的操作。

图2-15 HAL Interface 到 HAL 的打开摄像头流程图



3

参考文档

[1] <https://source.android.google.cn/devices/camera>